

Basic Abstractions in Distributed Algorithms I

Based on: Cachin, Guerraoui, Rodrigues

Danilo Kačanski

Faculty of Technical Sciences, Novi Sad
Applied algorithms in Control Systems

April 2026.

Framing the Model

Challenges of Distributed Systems

- **Real systems are inherently complex**
 - Many machines are interacting over a network.
 - Concurrent and parallel events can cause unpredictable execution order.
- **Networks are unreliable**
 - Messages may be delayed, lost, or duplicated.
 - Delivery and its order are not guaranteed.
- **Processes may fail**
 - Machines can crash unexpectedly.
 - Worse than that they may behave incorrectly or maliciously.

Key Challenge

Distributed algorithms must operate correctly despite all uncertainties above.

Why We Need Abstractions

- **Distributed systems require formal guarantees**
 - Safety must not depend on timing assumptions.
 - Algorithms must work under uncertainty.
- **Abstractions simplify complex systems**
 - They isolate essential properties of communication.
 - They hide unnecessary implementation details.
- **Abstractions enable formal reasoning**
 - Algorithms can be analyzed mathematically.
 - Because of that properties such as reliability and agreement become provable.

Key Idea

Chapter 2 from the book introduces the fundamental building blocks of distributed systems. Instead of implementing full systems immediately, we first want to understand abstractions that will allow us to reason about correctness and guarantees.

- **Single-threaded execution**

- The simulator executes one step at a time.
- This makes system behavior much easier to analyze.

- **No goroutines, no networking**

- We avoid real concurrency and definitely real networks.
- The goal for now is to model distributed behavior, not infrastructure.

- **Explicit nondeterminism**

- Message delivery order is chosen randomly.
- This will model asynchronous distributed execution without overcomplicating things at the start.

Process Model

State Machine Model

State Machine

A **state machine** is a computational model defined by:

- a set of **states** describing the internal configuration
- a set of **events or inputs**
- a **transition function** that determines how the state changes

Formally, the transition function is:

$$(state, input) \rightarrow (new\ state, outputs)$$

Distributed Process

In distributed algorithms, **process** is modeled as a state machine:

$$(current\ state, message) \rightarrow (new\ state, outgoing\ messages)$$

Process Interface (process/process.go)

Process Structure

- ID() - returns the process identifier
- Handle(msg) - processes an incoming message

Process Interface

```
type Process interface {  
    ID() string  
    Handle(msg Message) []Message  
}
```

Important

A **thread** is an operating system execution unit.

A **process** in the distributed model is an **abstract state machine** that can be implemented by a thread, goroutine, or a machine in a real system.

Message Abstraction (process/message.go)

- Processes communicate by sending **messages**.
- A message contains:
 - sender identifier
 - destination identifier
 - message type
 - optional payload data

Message Structure

```
type Message struct {  
    From ProcessID  
    To    ProcessID  
    Type  string  
    Data  any  
    Meta  map[string]any  
}
```

Key Operations

- `NewMessage(from, to, type, data)`: creates a new message with initialized metadata
- `String()`: returns human-readable representation (for logging and debugging)
- `ContentString()`: returns canonical content representation (for hashing, MAC, signatures)
- `Clone()`: creates an independent copy of the message to avoid shared state issues

Process State (process/state.go)

- Each process maintains its own **local state**.
- State is:
 - **Private**: no other process can access it
 - **Volatile**: lost on crash
 - **Updated atomically** during each Handle() step

State Structure

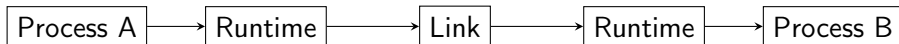
```
type State struct {  
    store map[string]any  
}
```

Key Operations

- `Set(key, value)`: stores a value
- `Get(key)`: retrieves value (may return nil if key is nonexistent)
- `GetInt` / `GetString` / `GetBool`: typed access so it is not checked manually non-stop
- `Has(key)`: checks existence of a key
- `Delete(key)`: removes value
- `Clear()`: resets state to simulate crash

All Communication Is Mediated

- Processes **never call each other directly**.
 - **In real system:** communication happens over network (TCP/UDP, RPC)
 - **Here:** processes return messages and runtime handles delivery
- Message delivery is controlled by the **runtime system**.
 - **In real system:** OS + network stack decide delivery timing
 - **Here:** runtime simulates network, links, and failures
- The scheduler decides **when and which message is delivered**.
 - **In real system:** asynchronous, unpredictable delivery order
 - **Here:** random scheduler simulates nondeterminism (reproducible via seed)



Link Hierarchy

Why Abstract Links?

- Real networks are **unreliable**:
 - loss, delay, duplication, reordering
- Algorithms require **clear guarantees**
- We build guarantees **on top of the network**:
 - retransmission, deduplication, authentication so we can counter that unreliableness
- Guarantees are **composed**:
 - Fair-Loss $\rightarrow$ Stubborn $\rightarrow$ Perfect $\rightarrow$ Authenticated

Key Idea

We do not use the raw networkm, instead we build abstractions that provide **clean guarantees** for designing and proving algorithms.

Fair-Loss Link (link/fairloss.go)

- The **baseline communication abstraction**.
- Models the behavior of a **real unreliable network**.
- Messages may be: lost, delayed and duplicated

Properties (Cachin et al.)

- **FLL1 (Fair-loss delivery)**: infinite sends $\Rightarrow$ infinite delivery
- **FLL2 (Finite duplication)**: finite sends $\Rightarrow$ finite deliveries
- **FLL3 (No creation)**: no fake messages are introduced

Key Idea

Fair-loss links provide **minimal guarantees**. Stronger links are built on top of this one.

Fair-Loss Link Key Operations

Key Operations

- `Send(msg, rng)`: may drop the message based on `LossRate`
- `Tick(step, rng)`: does nothing (no retransmission for fll)
- `Receive(msg)`: always accepts message (no filtering or checks)

Behavior in Simulator

- Each send has probability `LossRate` of being dropped
- Lost messages are simply **not enqueued**
- Reliability is achieved only through **repeated sends**

Stubborn Link (link/stubborn.go)

- Built on top of the **Fair-Loss Link**.
- Ensures delivery through **repeated retransmission**.
- Messages are **never forgotten** once sent.

Properties (Cachin et al.)

- **SL1 (Stubborn delivery)**: one send $\Rightarrow$ infinite deliveries
- **SL2 (No creation)**: no fake messages are introduced

Key Idea

Reliability is achieved by **sending messages again and again** over an unreliable channel until they get through.

Stubborn Link Key Operations

Key Operations

- `Send(msg, rng)`: stores message and forwards it via fair-loss link
- `Tick(step, rng)`: periodically retransmits **ALL stored messages**
- `Receive(msg)`: always accepts (duplicates are allowed)

Behavior in Simulator

- Every sent message is stored in a **buffer**
- On each retransmission step:
 - all stored messages are sent again
 - each send still goes through **fair-loss** (may fail)
- Delivery is guaranteed only through **infinite retries**
- Duplicate deliveries are **expected and allowed**

Perfect Link (link/perfect.go)

- Built on top of the **Stubborn Link**.
- Provides the communication abstraction **most algorithms actually want**: messages are eventually delivered, duplicates are suppressed and no fake messages are created
- This is achieved by combining:
 - **retransmission** from stubborn links
 - **deduplication** at receive time

Properties (Cachin et al.)

- **PL1 (Reliable delivery)**: one send $\Rightarrow$ eventual delivery
- **PL2 (No duplication)**: each message is delivered at most once
- **PL3 (No creation)**: no fake messages are introduced

Key Idea

Perfect Link = **Stubborn Link** + **Deduplication**

Perfect Link Key Operations

Key Operations

- `Send(msg, rng)`: forwards the message through the stubborn link
- `Tick(step, rng)`: delegates retransmission to the stubborn link
- `Receive(msg)`: checks whether the message was already delivered
 - if already seen $\Rightarrow$ suppress duplicate
 - if new $\Rightarrow$ mark as delivered and accept

Behavior in Simulator

- Reliability still comes from **repeated retransmission**
- Correctness is added by storing delivered message identifiers in:
 - `delivered map[string]bool`
- `messageKey(msg)` creates a unique key used for deduplication

Authenticated Perfect Link (link/authenticated.go)

- Built on top of the **Perfect Link**.
- Adds **authenticity** to reliable communication, so it prevents **forging messages**.

Property (Cachin et al.)

- **AL1 (Authenticity)**: if process q delivers a message with sender p , and p is correct, then that message was actually sent by p

How it works

- On send: attach a **MAC** to the message
- On receive: verify the MAC before delivery
- If the content was modified, verification fails and the message is rejected

Key Idea

Authenticated Perfect Link = **Perfect Link** + **MAC verification**

Authenticated Perfect Link Key Operations

Key Operations

- `Send(msg, rng)`:
 - clones the message
 - computes MAC over `ContentString()` and stores MAC in `Meta["mac"]`
 - forwards via perfect link
- `Tick(step, rng)`:
 - delegates to the underlying perfect link
- `Receive(msg)`:
 - first runs perfect-link deduplication
 - then checks whether a valid MAC exists and rejects the message if verification fails

Behavior in Simulator

- Reliability and no-duplication come from the **PL** and authenticity from the **MAC check**
- Metadata carries the MAC, but the MAC is computed only over the **logical content**